

=====

Spring Boot

=====

=> Spring framework available in market from 2004 onwards...

=> Spring Boot came into market in the year of 2015...

=> Spring Boot is an extension for spring framework.

=> Spring Boot is an approach to develop spring based applications with less configurations.

=> Using springboot we can develop several types of applications.

- 1) stand-alone apps
- 2) web apps
- 3) microservices

=> If we develop project using spring then we need to manage configurations on our own.

=> If we develop project using springboot then we will get Auto Configuration.

=> Using springboot we can achieve rapid application development.

=====

Advantages with springboot

=====

- 1) pom starters
- 2) Auto Configuration
- 3) Embedded Containers
- 4) Actuators

pom starters (dependencies) :: Simplifying dependencies configuration in pom.xml

```
** data-jpa-starter
** web-starter
** security-starter
** mail-starter
** actuator-starter
```

Note: pom starters are used to enable auto-configurations in boot application.

Auto Configuration :: Based on pom starters boot will identify configurations required for the project and boot will manage those configurations in the runtime.

```
** spring-boot-starter -----> It will start IOC container
** data-jpa-starter -----> It will create connection pool
** web-starter -----> It will start tomcat server
** security-starter ---> Default Login page for Authentication
```

Embedded Servers/Containers :: Springboot will take care of servers to run our web applications (we no need to download & install)

=> Springboot will support 3 embedded containers

- 1) tomcat (default)
- 2) jetty

3) netty (web-flux)

Actuators :: Production Ready Features

=> Actuators are used to monitor and manage our applications.

```

** Check Health
** Check beans loaded
** Check URL Patterns
** Check Metrics
** Thread Dump
** Heap Dump
** Env Props

```

```

=====
How to install STS IDE
=====

```

1) download sts ide jar file

STS Download Link : https://cdn.spring.io/spring-tools/release/STS4/4.24.0.RELEASE/dist/e4.32/spring-tool-suite-4-4.24.0.RELEASE-e4.32.0-win32.win32.x86_64.self-extracting.jar

2) run that jar file from cmd

syntax : java -jar <file-name>

3) Go to sts folder and open "SpringToolSuite4.exe" file

```

=====
How to install IntelliJ IDE
=====

```

1) Download IntelliJ community version (free of cost)

Download Link : <https://www.jetbrains.com/idea/download/download-thanks.html?platform=windows&code=IIC>

2) Once .exe file is downloaded double click on that and install it.

```

=====
How to create springboot project ?
=====

```

Approach-1 : Create boot application using "spring initializer website" then download it and extract it and import into "Eclipse / STS / intelliJ" IDE.

URL : start.spring.io

Approach-2 : Use STS ide to create springboot application directley.

Note-1 : In Eclipse we can't create springboot project directley

Note-2 : In IntelliJ Community Edition (Free) we can't create springboot project directley.

Note-2 : In IntelliJ Enterprise Edition (Licensed) we can create springboot project directley.

```

=====
Springboot project folder structure
=====

```

sb-app

- src/main/java -----> project source code folder
 - ** Application.java -----> start/main class of springboot app (entry point)
- src/main/resources -----> config files (xml, ymls, props)
 - ** application.properties --> config properties (datasource, server-port ..)
- src/test/java -----> Unit test classes (JUnit)
 - ** ApplicationTest.java ---> JUnit class for unit testing
- Maven Dependencies -----> downloaded jars
- target -----> .class files will be created here
- pom.xml -----> Maven Config file

=====
 What is spring-boot-starter-parent ?
 =====

=> The "spring-boot-starter-parent" is a special starter in Spring Boot projects.

=> parent-starter will provide below functionalities

- 1) Default Configuration
- 2) Manages Dependency versions
- 3) Reduce boiler plate code

=====
 How to change embedded server port number ?
 =====

=> SpringBoot embedded server runs by default on 8080 port.

=> By adding "server.port" property in application.properties file we can change port number

server.port=9090

=====
 What is dev-tools in springboot ?
 =====

=> Devtools is a maven dependency that we can add in springboot app pom.xml file..

=> It is used to auto restart our embedded servers when there is a code change in our application.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

=====
 What is start class in springboot ?
 =====

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

=> This start class will be created by default when boot app got created.

=> Start class is also called as main class of springboot application.

=> It is entrypoint for boot application execution. Execution will start from here only.

=====
Q) What is @SpringBootApplication annotation ?
=====

=> This is used at start class of the springboot.

=> This annotation is equal to below 3 annotations

- 1) @SpringBootConfiguration
- 2) @EnableAutoConfiguration
- 3) @ComponentScan

@SpringBootConfiguration : Indicates that this class provides Spring Boot application configuration.

@EnableAutoConfiguration : Tells Spring Boot to automatically configure your application based on the dependencies you have in your classpath.

@ComponentScan : Automatically discovers and registers spring beans available in the application.

Note: SpringBoot will consider start class package as base package.

=====
Q) What is component scanning and how it works internally ?
=====

=> It is the process of identifying spring beans available in the project.

=> Component Scanning will start from base package (the package which contains start class).

=> Once base package scanning completed, then it will go for sub packages of base package.

Note: Any package name which is starting with base package name is called as sub package.

in.ashokit ----- (base package)

in.ashokit.beans ----- will be scanned

in.ashokit.dao ----- will be scanned

in.ashokit.service ----- will be scanned

com.tcs.util ----- will not be scanned

Note: We can configure more than one base package using @ComponentScan annotation like below.

```
@SpringBootApplication
@ComponentScan(basePackages = { "in.ashokit", "com.tcs" })
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

=====
How Start class run () method works internally ?

=====

- start the timer
- create bootstrap context
- load listener classes
- prepare environment (read props/yml file)
- print banner
- create application-context (ioc)
- prepare and refresh IOC (DI)
- stop the timer
- calculate time taken to start application and print on console
- call runners
- return IOC obj

=====

Q) How IOC container will be started in Springboot application ?

=====

=> run () method will take care of starting IOC container.

=> run () method will use below classes to start IOC container based on starter available in pom.xml

spring-boot-starter :: AnnotationConfigApplicationContext

spring-boot-starter-web :: AnnotationConfigServletWebServerApplicationContext

spring-boot-starter-webflux :: AnnotationConfigReactiveWebServerApplicationContext

...

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);
        System.out.println("Class Name :: " + context.getClass().getName());
    }
}
```

Note-1: when we use "web-starter" boot will give "tomcat" as default embedded container.

Note-2: When we use "webflux-starter" boot will give "netty" as default embedded container.

=====

What is banner in springboot ?

=====

=> When we run boot application by default spring logo will be printed on console that is called as banner in springboot.

Note: run () method contains logic to print the banner.

=> springboot banner works based on modes. We have 3 types of modes here

- 1) console (default)

2) log

3) off

=> We can change banner mode using below property in "application.properties" file

```
spring.main.banner-mode=off
```

```
=====
What is runner in springboot ?
=====
```

-> Runners are used to execute any logic only once when the boot application got started.

-> Runners will be called by SpringApplication.run () method.

use cases:

1) To delete temporary tables data when the app got started

2) Load static tables data into cache memory when app got started

=> We have 2 types of runners in springboot

1) ApplicationRunner (FI) ==> run(..)

2) CommandLineRunner (FI) ==> run(..)

```
-----
@Component
public class MyAppRunner implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) throws Exception {
        System.out.println("AppRunner executed...");
    }
}
-----
```

```
@Component
public class MyCmdRunner implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        System.out.println("MyCmdRunner executed...");
    }
}
-----
```

```
$ java -jar app.jar --mode=test --x=5 --table=product user1 user2
```

```
-----
CommandLineRunner : receiving raw command-line arguments.
-----
```

```
Args: --mode=test
Args: --count=5
Args: user1
Args: user2
```

```
-----
ApplicationRunner : parsed arguments, wrapped inside the ApplicationArguments object
-----
```

```
Option names: [mode, count]
Non-option args: [user1, user2]
```